

Newton's Method

You can use numerical techniques to find your global maximum by following the reasoning underlying **Newton's method from calculus**; namely, that you can pretty well approximate a function with the first few terms of its **Taylor Series**.

Specifically, given a step size γ , you compute a local gradient $\nabla \Theta$, which corresponds to the first derivative, and a Hessian matrix H , which corresponds to the second derivative. You put them together and each step of the algorithm looks something like this:

$$\Theta_{n+1} = \Theta_n - \gamma H^{-1} \cdot \nabla \Theta.$$

Newton's method uses the curvature of log-likelihood to choose an appropriate step direction. Note that this calculation involves inverting the $k \times k$ Hessian matrix, which is bad when there's lots of features, as in 10,000 or something. Typically you don't have that many features, but it's not impossible.

In practice you'd never actually invert the Hessian—instead you'd solve an equation of the form $Ax = y$, which is much more computationally stable than finding A^{-1} .

Stochastic Gradient Descent

Another possible method to maximize your likelihood (or minimize your negative log likelihood) is called **Stochastic Gradient Descent**. It approximates a gradient using a single observation at a time. The algorithm updates the current best-fit parameters each time it sees a new data point. The good news is that there's no big matrix inversion, and it works well with both huge data and sparse features; it's a big deal in **Mahout** and **Vowpal Wabbit**, two open source projects to enable large-scale machine learning across a variety of algorithms. The bad news is it's not such a great optimizer and it's very dependent on step size.

Implementation

In practice you don't have to code up the iterative reweighted least squares or stochastic gradient optimization method yourself; this is implemented in R or in any package that implements logistic regression. So suppose you had a dataset that had the following first five rows:

click	url_1	url_2	url_3	url_4	url_5
1	0	0	0	1	0
1	0	1	1	0	1
0	1	0	0	1	0
1	0	0	0	0	0
1	1	0	1	0	1

Call this matrix “train,” and then the command line in R would be:

```
fit <- glm(click ~ url_1 + url_2 + url_3 + url_4 + url_5,
           data = train, family = binomial(logit))
```

Evaluation

Let’s go back to the big picture from earlier in the chapter where we told you that you have many choices you need to make when confronted with a classification problem. One of the choices is how you’re going to evaluate your model. We discussed this already in [Chapter 3](#) with respect to linear regression and k-NN, as well as in the previous chapter with respect to Naive Bayes. We generally use different evaluation metrics for different kinds of models, and in different contexts. Even logistic regression can be applied in multiple contexts, and depending on the context, you may want to evaluate it in different ways.

First, consider the context of using logistic regression as a ranking model—meaning you are trying to determine the *order* in which you show ads or items to a user based on the probability they would click. You could use logistic regression to estimate probabilities, and then rank-order the ads or items in decreasing order of likelihood to click based on your model. If you wanted to know how good your model was at discovering relative rank (notice in this case, you could care less about the absolute scores), you’d look to one of:

Area under the receiver operating curve (AUC)

In signal detection theory, a receiver operating characteristic curve, or ROC curve, is defined as a plot of the true positive rate against the false positive rate for a binary classification problem as you change a threshold. In particular, if you took your training set and ranked the items according to their probabilities and varied the threshold (from ∞ to $-\infty$) that determined whether to classify the item as 1 or 0, and kept plotting the true positive rate versus the false positive rate, you’d get the ROC curve. The area under that curve, referred to as the AUC, is a way to measure the success of a classifier or to compare two classifiers. Here’s a nice paper on it by Tom Fawcett, “[Introduction to ROC Analysis](#)”.

Area under the cumulative lift curve

An alternative is the area under the cumulative lift curve, which is frequently used in direct marketing and captures how many times it is better to use a model versus not using the model (i.e., just selecting at random).

We'll see more of both of these in [Chapter 13](#).

Warning: Feedback Loop!

If you want to productionize logistic regression to rank ads or items based on clicks and impressions, then let's think about what that means in terms of the data getting generated. So let's say you put an ad for hair gel above an ad for deodorant, and then more people click on the ad for hair gel—is it because you put it on top or because more people want hair gel? How can you feed that data into future iterations of your algorithm given that you potentially caused the clicks yourself and it has nothing to do with the ad quality? One solution is to always be logging the *position* or *rank* that you showed the ads, and then use *that* as one of the predictors in your algorithm. So you would then model the probability of a click as a function of position, vertical, brand, or whatever other features you want. You could then use the parameter estimated for position and use that going forward as a “position normalizer.” There's a whole division at Google called Ads Quality devoted to problems such as these, so one little paragraph can't do justice to all the nuances of it.

Second, now suppose instead that you're using logistic regression for the purposes of classification. Remember that although the observed output was a binary outcome (1,0), the output of a logistic model is a probability. In order to use this for classification purposes, for any given unlabeled item, you would get its predicted probability of a click. Then to minimize the misclassification rate, if the predicted probability is > 0.5 that the label is 1 (click), you would label the item a 1 (click), and otherwise 0. You have several options for how you'd then evaluate the quality of the model, some of which we already discussed in [Chapters 3 and 4](#), but we'll tell you about them again here so you see their pervasiveness:

Lift

How much more people are buying or clicking because of a model (once we've introduced it into production).

Accuracy

How often the correct outcome is being predicted, as discussed in Chapters 3 and 4.

Precision

This is the (number of true positives)/(number of true positives + number of false positives).

Recall

This is the (number of true positives)/(number of true positives + number of false negatives).

F-score

We didn't tell you about this one yet. It essentially combines precision and recall into a single score. It's the harmonic mean of precision and recall, so $(2 \times \text{precision} \times \text{recall}) / (\text{precision} + \text{recall})$. There are generalizations of this that are essentially changing how much you weight one or the other.

Finally, for density estimation, where we need to know an actual probability rather than a relative score, we'd look to:

Mean squared error

We discussed with respect to linear regression. As a reminder, this is the average squared distance between the predicted and actual values.

Root squared error

The square root of mean squared error.

Mean absolute error

This is a variation on mean squared error and is simply the average of the absolute value of the difference between the predicted and actual values.

In general, it's hard to compare lift curves, but you can compare AUC (area under the receiver operator curve)—they are “base rate invariant.” In other words, if you bring the click-through rate from 1% to 2%, that's 100% lift; but if you bring it from 4% to 7%, that's less lift but more effect. AUC does a better job in such a situation when you want to compare.

Density estimation tests tell you how well are you fitting for conditional probability. In advertising, this may arise if you have a situation where each ad impression costs \$c and for each conversion you receive

$\$q$. You will want to target every conversion that has a positive expected value, i.e., whenever:

$$P(\text{Conversion}|X) \cdot \$q > \$c$$

But to do this you need to make sure the probability estimate on the left is *accurate*, which in this case means something like the mean squared error of the estimator is small. Note a model can give you good relative rankings—it gets the order right—but bad estimates of the probability.

Think about it this way: it could say to rank items in order 1, 2, and 3 and estimate the probabilities as .7, .5, and .3. It might be that the *true* probabilities are .03, .02, and .01, so our estimates are totally off, but the ranking was correct.

Using A/B Testing for Evaluation

When we build models and optimize them with respect to some evaluation metric such as accuracy or mean squared error, the estimation method itself is built to optimize parameters *with respect* to these metrics. In some contexts, the metrics we might want to optimize for are something else altogether, such as revenue. So we might try to build an algorithm that optimizes for accuracy, when our real goal is to make money. The model itself may not directly capture this. So a way to capture it is to run A/B tests (or statistical experiments) where we divert some set of users to one version of the algorithm and another set of users to another version of the algorithm, and check the difference in performance of metrics we care about, such as revenue or revenue per user, or something like that. We'll discuss A/B testing more in [Chapter 11](#).

Media 6 Degrees Exercise

Media 6 Degrees kindly provided a dataset that is perfect for exploring logistic regression models, and evaluating how good the models are. Follow along by implementing the following R code. The dataset can be found at https://github.com/oreillymedia/doing_data_science.

Sample R Code

```
# Author: Brian Dalessandro
# Read in data, look at the variables and create a training
and test set
file <- "binary_class_dataset.txt"
set <- read.table(file, header = TRUE, sep = "\t",
                  row.names = "client_id")

names(set)

split <- .65
set["rand"] <- runif(nrow(set))
train <- set[(set$rand <= split), ]
test <- set[(set$rand > split), ]
set$Y <- set$Y_BUY
```

```
#####
##### R FUNCTIONS #####
#####
```

```
library(mgcv)
```

```
# GAM Smoothed plot
```

```
plotrel <- function(x, y, b, title) {
  # Produce a GAM smoothed representation of the data
  g <- gam(as.formula("y ~ x"), family = "binomial",
          data = set)
  xs <- seq(min(x), max(x), length = 200)
  p <- predict(g, newdata = data.frame(x = xs),
              type = "response")

  # Now get empirical estimates (and discretize if
  non discrete)
  if (length(unique(x)) > b) {
    div <- floor(max(x) / b)
    x_b <- floor(x / div) * div
    c <- table(x_b, y)
  }
  else { c <- table(x, y) }
  pact <- c[, 2]/(c[, 1]+c[, 2])
  cnt <- c[, 1]+c[, 2]
  xd <- as.integer(rownames(c))
  plot(xs, p, type="l", main=title,
       ylab = "P(Conversion | Ad, X)", xlab="X")
  points(xd, pact, type="p", col="red")
  rug(x+runif(length(x)))
}
```

```
library(plyr)
```

```

# wMAE plot and calculation
getmae <- function(p, y, b, title, doplot) {
  # Normalize to interval [0,1]
  max_p <- max(p)
  p_norm <- p / max_p
  # break up to b bins and rescale
  bin <- max_p * floor(p_norm * b) / b
  d <- data.frame(bin, p, y)
  t <- table(bin)
  summ <- dplyr::ddply(d, .(bin), summarise, mean_p = mean(p),
    mean_y = mean(y))
  fin <- data.frame(bin = summ$bin, mean_p = summ$mean_p,
    mean_y = summ$mean_y, t)

  # Get wMAE
  num = 0
  den = 0
  for (i in c(1:nrow(fin))) {
    num <- num + fin$Freq[i] * abs(fin$mean_p[i] -
      fin$mean_y[i])
    den <- den + fin$Freq[i]
  }
  wmae <- num / den
  if (doplot == 1) {
    plot(summ$bin, summ$mean_p, type = "p",
      main = paste(title, " MAE =", wmae),
      col = "blue", ylab = "P(C | AD, X)",
      xlab = "P(C | AD, X)")
    points(summ$bin, summ$mean_y, type = "p", col = "red")
    rug(p)
  }
  return(wmae)
}

library(ROCR)
get_auc <- function(ind, y) {
  pred <- prediction(ind, y)
  perf <- performance(pred, 'auc', fpr.stop = 1)
  auc <- as.numeric(substr(slot(perf, "y.values"), 1, 8),
    double)
  return(auc)
}

# Get X-Validated performance metrics for a given feature set
getxval <- function(vars, data, folds, mae_bins) {
  # assign each observation to a fold
  data["fold"] <- floor(runif(nrow(data)) * folds) + 1
  auc <- c()
  wmae <- c()

```

```

fold <- c()
# make a formula object
f = as.formula(paste("Y", "~", paste(vars,
                                collapse = "+")))
for (i in c(1:folds)) {
  train <- data[(data$fold != i), ]
  test <- data[(data$fold == i), ]
  mod_x <- glm(f, data=train, family = binomial(logit))
  p <- predict(mod_x, newdata = test, type = "response")
  # Get wMAE
  wmae <- c(wmae, getmae(p, test$Y, mae_bins,
                        "dummy", 0))
  fold <- c(fold, i)
  auc <- c(auc, get_auc(p, test$Y))
}
return(data.frame(fold, wmae, auc))
}

#####
#####          MAIN: MODELS AND PLOTS          #####
#####
# Now build a model on all variables and look at coefficients
and model fit
vlist <-      c("AT_BUY_BOOLEAN",      "AT_FREQ_BUY",
"AT_FREQ_LAST24_BUY",
      "AT_FREQ_LAST24_SV", "AT_FREQ_SV", "EXPECTED_TIME_BUY",
      "EXPECTED_TIME_SV", "LAST_BUY", "LAST_SV", "num_checkins")
f = as.formula(paste("Y_BUY", "~" , paste(vlist,
                                collapse = "+")))
fit <- glm(f, data = train, family = binomial(logit))
summary(fit)

# Get performance metrics on each variable

vlist <-      c("AT_BUY_BOOLEAN",      "AT_FREQ_BUY",
"AT_FREQ_LAST24_BUY",
      "AT_FREQ_LAST24_SV", "AT_FREQ_SV", "EXPECTED_TIME_BUY",
      "EXPECTED_TIME_SV", "LAST_BUY", "LAST_SV", "num_checkins")

# Create empty vectors to store the performance/evaluation met
rics
auc_mu <- c()
auc_sig <- c()
mae_mu <- c()
mae_sig <- c()

for (i in c(1:length(vlist))) {
  a <- getxval(c(vlist[i]), set, 10, 100)
  auc_mu <- c(auc_mu, mean(a$auc))
  auc_sig <- c(auc_sig, sd(a$auc))
  mae_mu <- c(mae_mu, mean(a$wmae))
}

```

```

    mae_sig <- c(mae_sig, sd(a$wmae))
  }

  univar <- data.frame(vlist, auc_mu, auc_sig, mae_mu, mae_sig)

  # Get MAE plot on single variable -
  use holdout group for evaluation
  set <- read.table(file, header = TRUE, sep = "\t",
    row.names="client_id")
  names(set)

  split<-.65
  set["rand"] <- runif(nrow(set))
  train <- set[(set$rand <= split), ]
  test <- set[(set$rand > split), ]
  set$Y <- set$Y_BUY

  fit <- glm(Y_BUY ~ num_checkins, data = train,
    family = binomial(logit))
  y <- test$Y_BUY
  p <- predict(fit, newdata = test, type = "response")

  getmae(p,y,50,"num_checkins",1)

  # Greedy Forward Selection
  rvars <- c("LAST_SV", "AT_FREQ_SV", "AT_FREQ_BUY",
    "AT_BUY_BOOLEAN", "LAST_BUY", "AT_FREQ_LAST24_SV",
    "EXPECTED_TIME_SV", "num_checkins",
    "EXPECTED_TIME_BUY", "AT_FREQ_LAST24_BUY")
  # Create empty vectors
  auc_mu <- c()
  auc_sig <- c()
  mae_mu <- c()
  mae_sig <- c()

  for (i in c(1:length(rvars))) {
    vars <- rvars[1:i]
    vars
    a <- getxval(vars, set, 10, 100)
    auc_mu <- c(auc_mu, mean(a$auc))
    auc_sig <- c(auc_sig, sd(a$auc))
    mae_mu <- c(mae_mu, mean(a$wmae))
    mae_sig <- c(mae_sig, sd(a$wmae))
  }
  kvar<-data.frame(auc_mu, auc_sig, mae_mu, mae_sig)

  # Plot 3 AUC Curves
  y <- test$Y_BUY

  fit <- glm(Y_BUY~LAST_SV, data=train,
    family = binomial(logit))

```

```

p1 <- predict(fit, newdata=test, type="response")
fit <- glm(Y_BUY~LAST_BUY, data=train,
          family = binomial(logit))
p2 <- predict(fit, newdata=test, type="response")
fit <- glm(Y_BUY~num_checkins, data=train,
          family = binomial(logit))
p3 <- predict(fit, newdata=test,type="response")

pred <- prediction(p1,y)
perf1 <- performance(pred, 'tpr', 'fpr')
pred <- prediction(p2,y)
perf2 <- performance(pred, 'tpr', 'fpr')
pred <- prediction(p3,y)
perf3 <- performance(pred, 'tpr', 'fpr')

plot(perf1, color="blue", main="LAST_SV (blue),
      LAST_BUY (red), num_checkins (green)")
plot(perf2, col="red", add=TRUE)
plot(perf3, col="green", add=TRUE)

```

